МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра вычислительных технологий

ЛАБОРАТОРНАЯ РАБОТА №2

Дисциплина: ТЕХНОЛОГИИ СОЗДАНИЯ И ПОДДЕРЖКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Тема: «Проектирование программного обеспечения с использованием UML-диаграмм»

Работу выполнила: _____ Баннова.Д.А

Направление подготовки: 09.03.03 Прикладная информатика

Преподаватель: профессор кафедры анализа данных и искусственного интеллекта факультета компьютерных технологий и прикладной математики, к. техн. н., доцент

_____ Р.Ю. Вишняков

Краснодар

2026

Цель работы:

Изучение методологии объектно-ориентированного проектирования и получение практических навыков создания структурных и поведенческих UML-диаграмм. Детальный разбор связей, элементов и логики взаимодействия компонентов системы на примере интеллектуального планировщика задач Flowra.

Введение:

UML (Unified Modeling Language) — это стандартизированный графический язык для визуализации и документирования программных систем. В соответствии с теорией проектирования, UML-диаграммы делятся на две группы:

1. **Структурные диаграммы** (Class Diagram, Component Diagram) — показывают статическую архитектуру системы: из каких блоков она состоит и как они связаны друг с другом.
2. **Поведенческие диаграммы** (Use Case, Activity, Sequence, State Machine) — показывают динамику: как система работает, какие алгоритмы выполняет и как обменивается сообщениями во времени.

Ниже представлен детальный разбор архитектуры разрабатываемой системы Flowra.

ПРАКТИЧЕСКАЯ ЧАСТЬ

1. Поведенческая диаграмма: Диаграмма вариантов использования (Use Case Diagram)

Назначение диаграммы:

Диаграмма вариантов использования (прецедентов) описывает функциональное назначение разрабатываемой системы Flowra. Она показывает, что делает система с точки зрения внешнего наблюдателя, не вдаваясь в технические детали реализации.

Детальный разбор элементов и связей:

- **Акторы (Actors): Обозначены фигурками человечков. Это внешние сущности, взаимодействующие с системой. Главный актор — «Пользователь» (инициатор большинства действий). Вторичные (системные) акторы — «Telegram-бот», «Календарь» и «Платёжная система», которые выступают источниками или приемниками данных.**

- **Прецеденты (Use Cases): Овалы, обозначающие конкретные бизнес-задачи (например, «Сформировать план задач»).**
- **Отношение включения (<<include>>): Обозначено пунктирной стрелкой. Это строгая зависимость, означающая, что базовый прецедент не может быть выполнен без выполнения включаемого.**
    - ***Обоснование на схеме:* Прецедент «Добавить задачу через ИИ» (выделен фиолетовым) обязательно включает в себя скрытый от пользователя процесс «Распознать естественную речь». Без работы NLP-модуля добавление задачи свободным текстом невозможно.**
- **Отношение расширения (<<extend>>): Обозначает опциональное (условное) поведение. Расширяющий прецедент выполняется только при наступлении определенных условий.**
    - ***Обоснование на схеме:* Прецедент «Активировать Rescue Mode» (выделен красным) расширяет базовый процесс «Сформировать план задач». Он срабатывает не всегда, а только в случае, если система фиксирует критический перегруз пользователя (овербукинг). Стрелка направлена от расширяющего прецедента к базовому.**

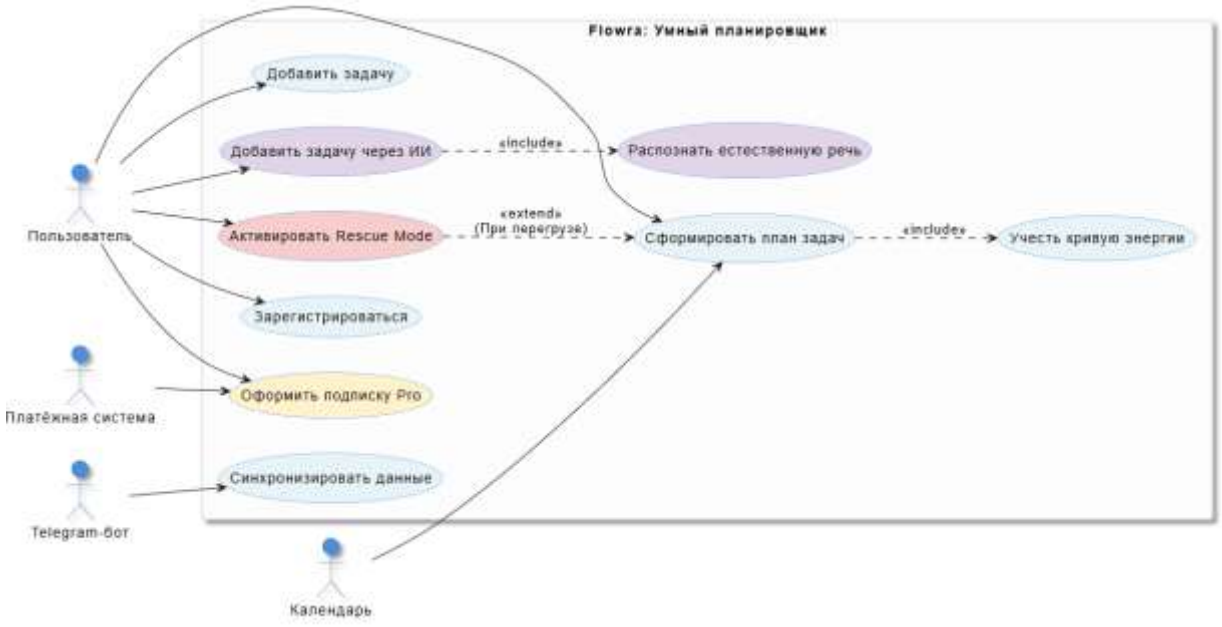


Рис. 1. Поведенческая диаграмма: Диаграмма вариантов использования

2. Структурная диаграмма: Диаграмма классов (Class Diagram)

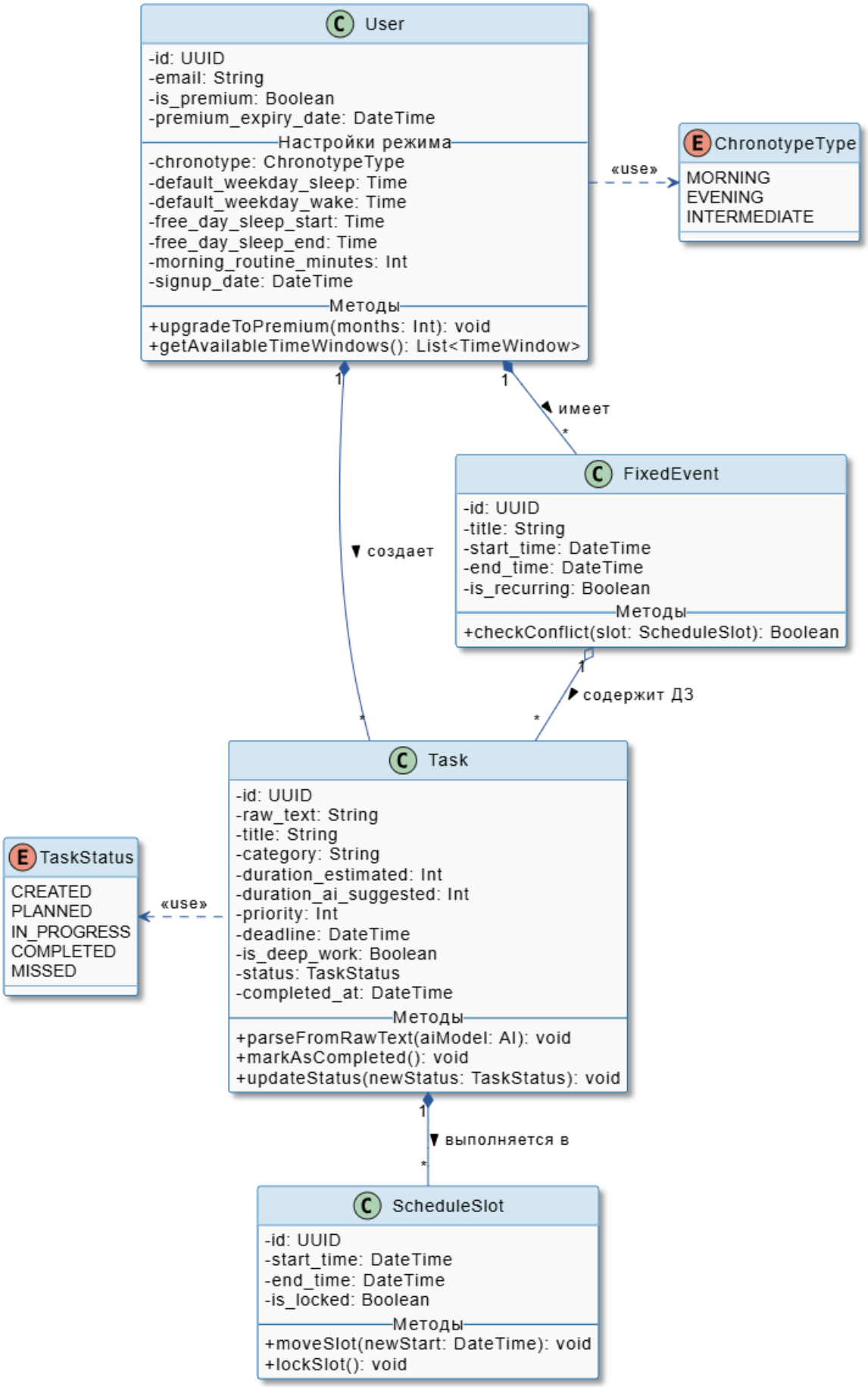


Рис. 2. Диаграмма классов (Class diagram) предметной области Flowra

Подробное описание классов, атрибутов и методов:

Диаграмма классов является основой программной архитектуры системы. На ней представлены главные сущности приложения Flowra, их инкапсулированные данные (поля) и поведение (методы).

1. Класс User (Пользователь)

Центральный класс системы, хранящий как идентификационные данные, так и сложные настройки персонализации для алгоритма планирования.

- **Атрибуты (private):**
  - id: UUID, email: String — базовые идентификаторы.
  - is_premium: Boolean, premium_expiry_date: DateTime — параметры биллинга (подписка Pro).
  - *Блок настроек режима:* chronotype (ссылается на Enum-перечисление), default_weekday_sleep / default_weekday_wake (время сна в будни), free_day_sleep_start / free_day_sleep_end (время сна в выходные), morning_routine_minutes (время на утренние сборы). Эти поля критически важны для "умного" распределения задач.
- **Методы (public):**
  - upgradeToPremium(months: Int) — функция продления подписки.
  - getAvailableTimeWindows(): List<TimeWindow> — сложный вычислительный метод, который вычитает из 24 часов время сна и рутины, возвращая чистые "окна", куда алгоритм может ставить задачи.

2. Класс Task (Задача)

Сущность, описывающая конкретное дело пользователя.

- **Атрибуты:**
  - raw_text: String — исходный запрос пользователя (например, введенный голосом).
  - title, category, priority, deadline — классические параметры задачи.
  - duration_estimated: Int (оценка пользователя) и duration_ai_suggested: Int (оценка от нейросети) — разделение этих полей позволяет системе самообучаться на разнице ожиданий и реальности.

    - is_deep_work: Boolean — флаг "глубокой работы". Алгоритм будет ставить такие задачи на пик активности хронотипа.
    - status: TaskStatus — текущее состояние задачи (ссылка на Enum).
- **Методы:**
    - parseFromRawText(aiModel: AI) — метод, который отправляет raw_text в ИИ-модуль и заполняет остальные поля.
    - markAsCompleted(), updateStatus() — методы управления жизненным циклом задачи.

3. Класс FixedEvent (Фиксированное событие)

Жесткие временные рамки (пары в университете, тренировки).

- **Атрибуты:** title, start_time, end_time, is_recurring (повторяющееся ли событие).
- **Методы:** checkConflict(slot: ScheduleSlot): Boolean — проверяет, не накладывается ли алгоритм планирования на это жесткое событие.

4. Класс ScheduleSlot (Слот расписания)

Конкретный отрезок времени на временной шкале календаря, выделенный под выполнение задачи.

- **Атрибуты:** start_time, end_time, is_locked (флаг закрепления слота пользователем, чтобы алгоритм его не сдвинул при пересчете).
- **Методы:** moveSlot(), lockSlot().

5. Перечисления (Enums)

Используются для строгой типизации, исключающей ошибки в коде:

- TaskStatus: CREATED, PLANNED, IN_PROGRESS, COMPLETED, MISSED.
- ChronotypeType: MORNING (Жаворонок), EVENING (Сова), INTERMEDIATE (Голубь).

Анализ связей между классами:

- **Композиция (Закрашенный ромб):** User жестко связан с Task и FixedEvent (создает и имеет их). Жизненный цикл подчинен: при удалении аккаунта пользователя каскадно удаляются все его задачи. Аналогично Task состоит из ScheduleSlot (задача может выполняться в несколько подходов/слотов).

- **Агрегация (Пустой ромб):** Связь между FixedEvent и Task. Задача (ДЗ) может быть логически привязана к расписанию пар, но при удалении задачи сама пара из расписания не исчезнет.

3. Поведенческая диаграмма: Диаграмма деятельностей (Activity Diagram)

Назначение диаграммы: Моделирует алгоритм работы системы. В данном случае визуализирован процесс автогенерации расписания.

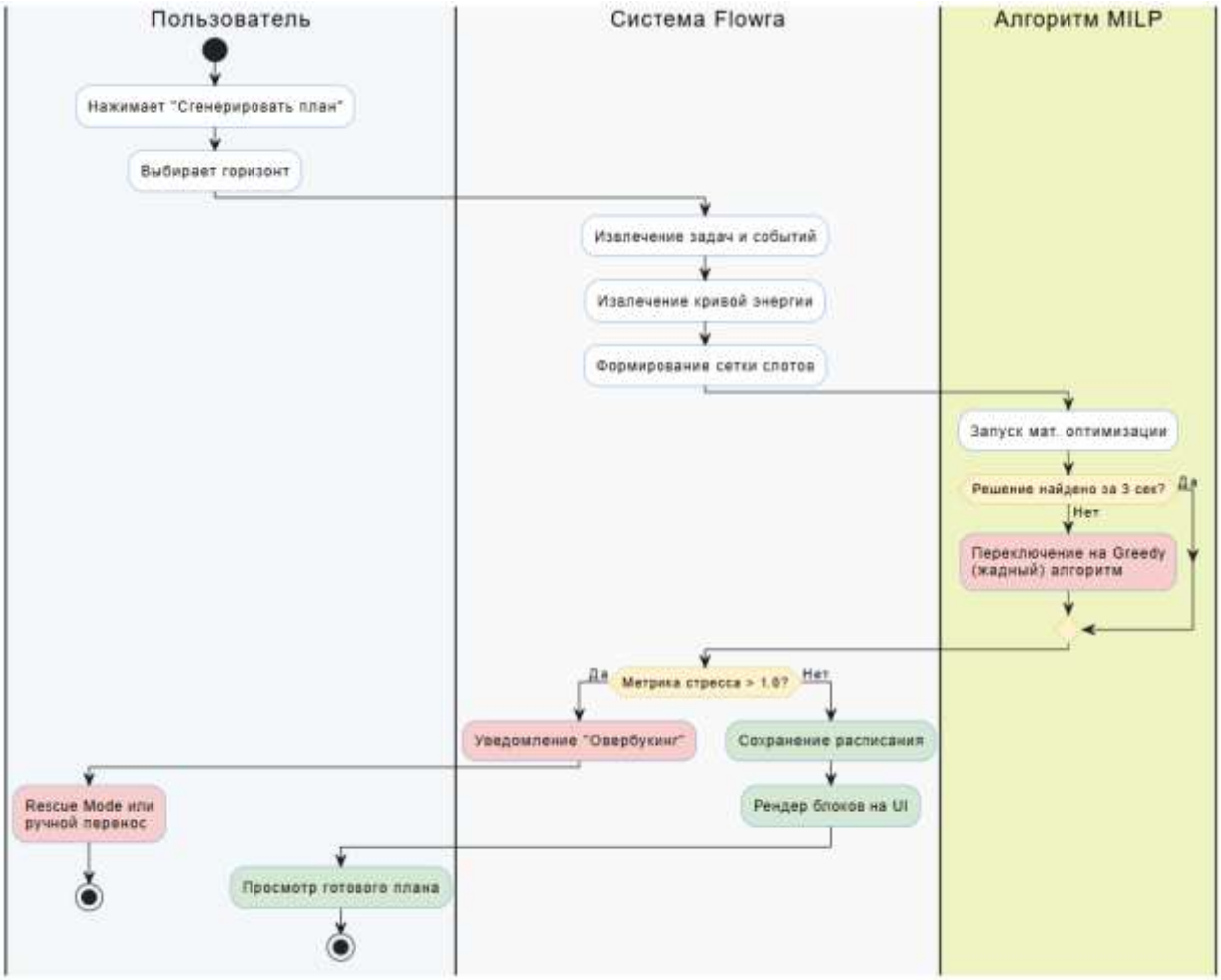


Рис. 3. Диаграмма деятельностей (Activity diagram) процесса автоматической генерации расписания

Диаграмма визуально разделена на три вертикальные колонки (их называют "свимлайны" или дорожки ответственности). Это сделано, чтобы сразу было понятно, *кто именно* выполняет действие на каждом шаге.

- **Шаг 1. Действия Пользователя (Левая колонка):**
    - Пользователь нажимает кнопку «Сгенерировать план» в приложении.

- Выбирает период (горизонт): распланировать только на сегодня или сразу на неделю.

- **Шаг 2. Подготовка данных (Средняя колонка — Система Flowra):**
  - Система лезет в базу данных и достает оттуда все невыполненные задачи пользователя.
  - Достает «железные» события (например, пары в университете, которые нельзя двигать).
  - Достает личную «Кривую энергии» (информацию о том, когда этот конкретный человек работает эффективнее всего — утром или вечером).
  - На основе этих данных система рисует "пустую сетку" свободных часов.
- **Шаг 3. Математика и планирование (Правая колонка — Алгоритм MILP):**
  - Данные передаются в **MILP** (Mixed-Integer Linear Programming — алгоритм математической оптимизации). Это сложный калькулятор, который пытается найти *идеальное* расположение задач.
  - **Желтый ромб (Условие 1):** У алгоритма есть ровно 3 секунды. Если за 3 секунды идеальный план найден — идем дальше. Если задача оказалась слишком сложной (закончилось время), алгоритм переключается на **Greedy (Жадный) алгоритм**. Это упрощенный метод планирования: он расставляет задачи быстро, заполняя первые попавшиеся дырки в расписании (не так идеально, зато без зависания телефона).
- **Шаг 4. Проверка на выгорание (Средняя колонка):**
  - **Желтый ромб (Условие 2 - Метрика стресса):** Система проверяет, реально ли вообще выполнить все задачи. Если задач на 10 часов, а свободного времени до сна осталось 5 часов (метрика стресса > 1.0), система идет по красной ветке.
  - **Красная ветка (Овербукинг):** Система честно говорит пользователю: "Ты не успеешь". Пользователю предлагается либо перенести дела руками, либо нажать кнопку **Rescue Mode** (Режим спасения), чтобы система сама выкинула неважные дела на завтра. Процесс останавливается.

- **Зеленая ветка (Успех):** Если время позволяет, система сохраняет готовое расписание в базу данных, рисует красивые цветные блоки в интерфейсе приложения, и пользователь видит свой готовый план на день.

4. Поведенческая диаграмма: Диаграмма последовательностей (Sequence Diagram)

Назначение диаграммы:
Эта схема показывает общение между частями программы **строго во времени** (хронологически сверху вниз). Мы рассматриваем конкретный сценарий: как система понимает текст вроде *"напомни сделать реферат по истории в пятницу, это займет часа 3"*.

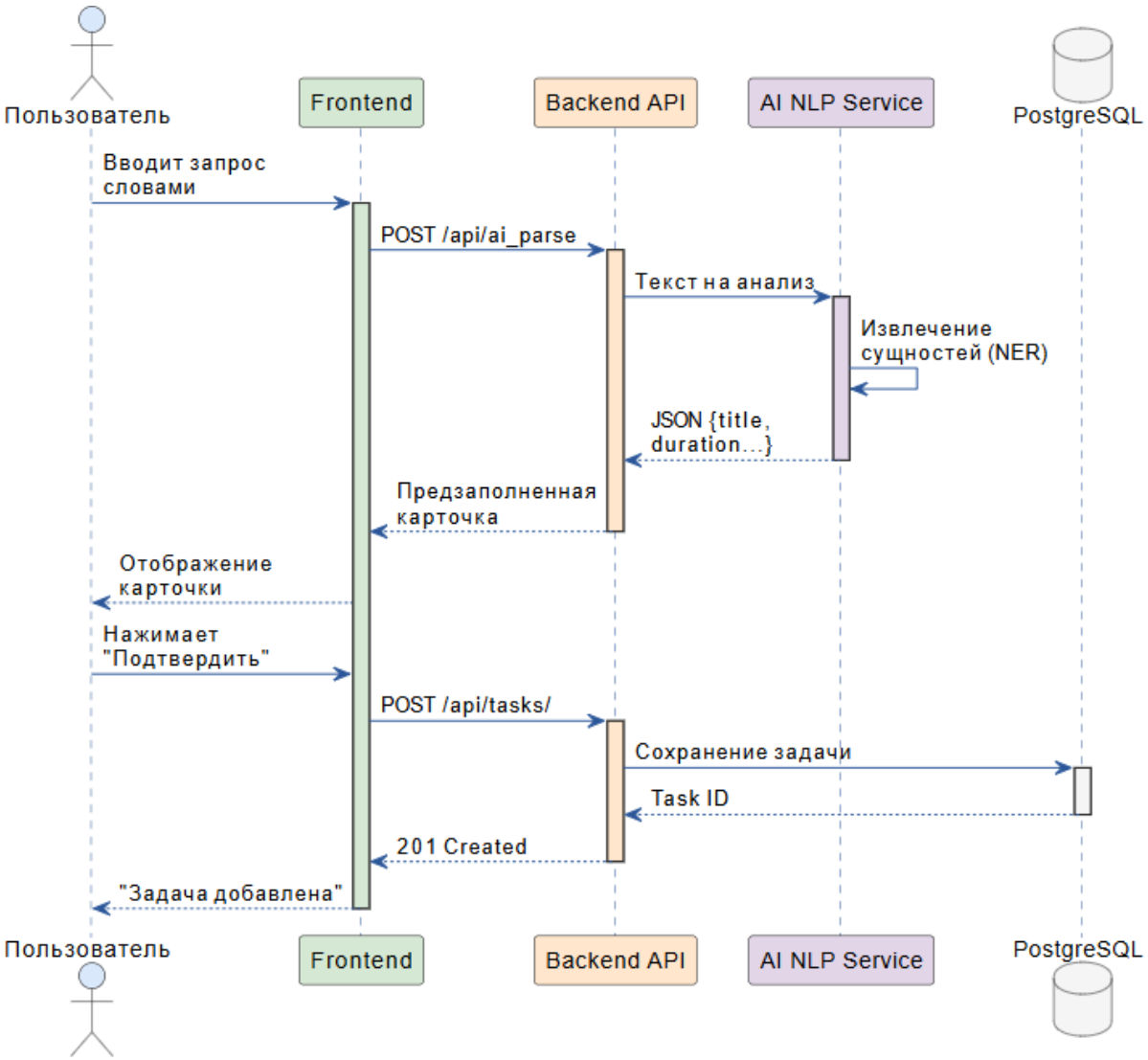


Рис. 4. Диаграмма последовательностей (Sequence diagram) процесса добавления задачи через ИИ

Расшифровка терминов (кто участники процесса):

- **Frontend (Фронтенд):** Это то, что видит пользователь (экран телефона, кнопочки, текстовые поля).
- **Backend API (Бэкенд):** "Мозг" приложения, скрытый на сервере. Он принимает команды от телефона и решает, что с ними делать.
- **AI NLP Service (ИИ-сервис):** Нейросеть (NLP — обработка естественного языка). Умная программа, которая умеет читать человеческий текст и извлекать из него смысл.
- **PostgreSQL:** База данных. Надежное хранилище, похожее на огромную Excel-таблицу, где всё сохраняется навсегда.

Пошаговый разбор процесса во времени:

1. **Пользователь** пишет в **Фронтенд** (в телефон) свободный текст: *"Сделать реферат..."*.
2. **Фронтенд** отправляет этот текст по интернету в **Бэкенд** (на сервер). Стрелка сплошная — телефон ждет ответа.
3. **Бэкенд** понимает, что это не обычная задача, и пересылает текст в **Нейросеть (AI)**.
4. **Нейросеть** читает текст и извлекает из него конкретику: Название ("Реферат"), Длительность ("3 часа"), Дедлайн ("Пятница").
5. Нейросеть упаковывает это в **JSON** (простой текстовый формат данных, понятный компьютерам) и возвращает **Бэкенду**.
6. **Бэкенд** пересылает эти данные в телефон (**Фронтенд**).
7. **Фронтенд** рисует на экране телефона красиво заполненную карточку задачи. Пользователь проверяет, правильно ли ИИ всё понял, и нажимает кнопку "Подтвердить".
8. **Фронтенд** говорит **Бэкенду**: "Всё верно, сохраняй".
9. **Бэкенд** отправляет команду в базу данных **PostgreSQL**: "Запиши новую задачу на жесткий диск".
10. База данных отвечает: "Успешно, вот номер (ID) новой задачи".
11. **Бэкенд** шлет телефону ответ "201 Created" (код успешного создания).
12. **Фронтенд** показывает пользователю зеленую галочку: "Задача добавлена". Процесс завершен.

5. Поведенческая диаграмма: Диаграмма состояний (State Machine Diagram)

Назначение диаграммы:
Показывает "жизненный путь" одной конкретной Задачи. Как она рождается, через какие статусы проходит и как "умирает" (завершается).

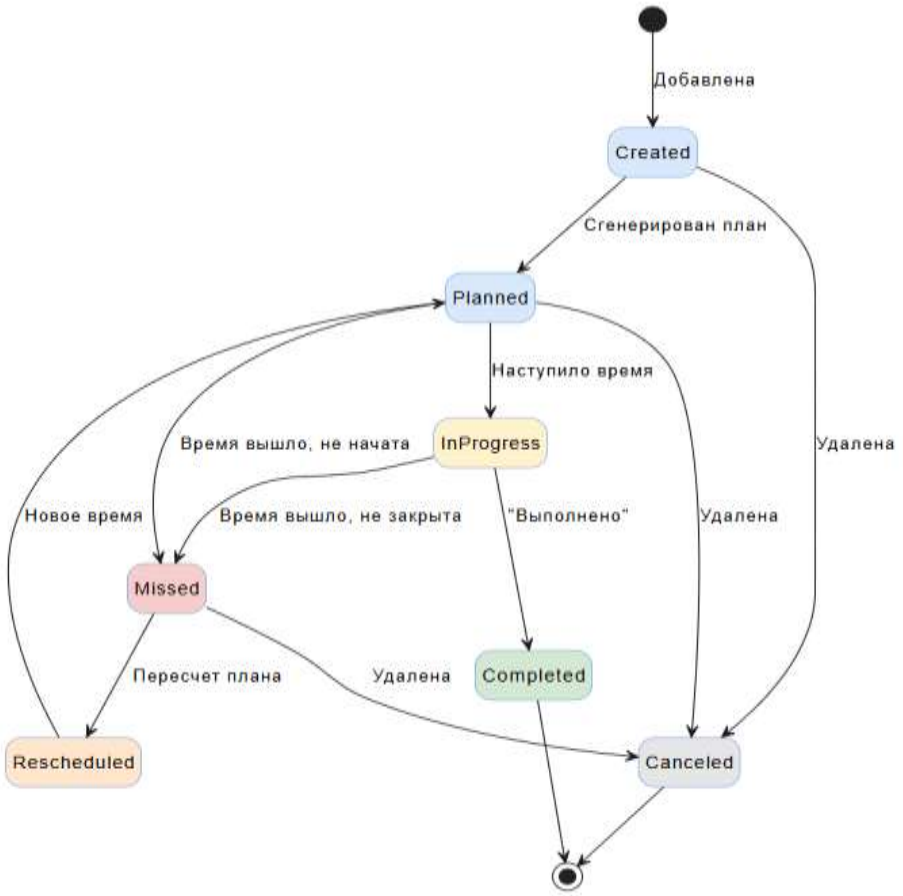


Рис. 5. Диаграмма состояний (State Machine Diagram)

Пошаговый разбор жизненного цикла задачи:

- **Рождение (Черный кружок сверху):**
  - Пользователь вводит задачу. Она получает статус **Created** (Создана).
  - Как только алгоритм нашел для нее место в расписании, она переходит в статус **Planned** (Запланирована).
- **Счастливый путь (Happy Path):**
  - На часах наступает время выполнения. Задача переходит в статус **InProgress** (В процессе).
  - Пользователь выполняет дело и нажимает галочку. Задача переходит в зеленый статус **Completed** (Выполнена) и достигает кружка с точкой (Смерть/Архив).

- **Проблемный путь (Что-то пошло не так):**
    - Наступил вечер, а пользователь даже не начал задачу, или начал, но не закончил. Срабатывает таймер, и задача падает в красный статус **Missed** (Пропущена).
    - Система не бросает пользователя. Запускается механизм пересчета, задача получает
      статус **Rescheduled** (Перепланирована) и возвращается обратно в голубой статус **Planned**, но уже на другой день.
- **Отмена:**
    - Из любого рабочего состояния (Создана, Запланирована, Пропущена) пользователь может нажать кнопку "Удалить". Задача уходит в статус **Canceled** (Отменена) и процесс завершается.

6. Структурная диаграмма: Диаграмма компонентов (Component Diagram)

Назначение диаграммы:
Это чертеж физического устройства приложения. Если программисты будут настраивать сервера для приложения Flowra, они будут смотреть именно на эту схему. Она объясняет, какие программы нужно установить и как они общаются друг с другом по интернету.

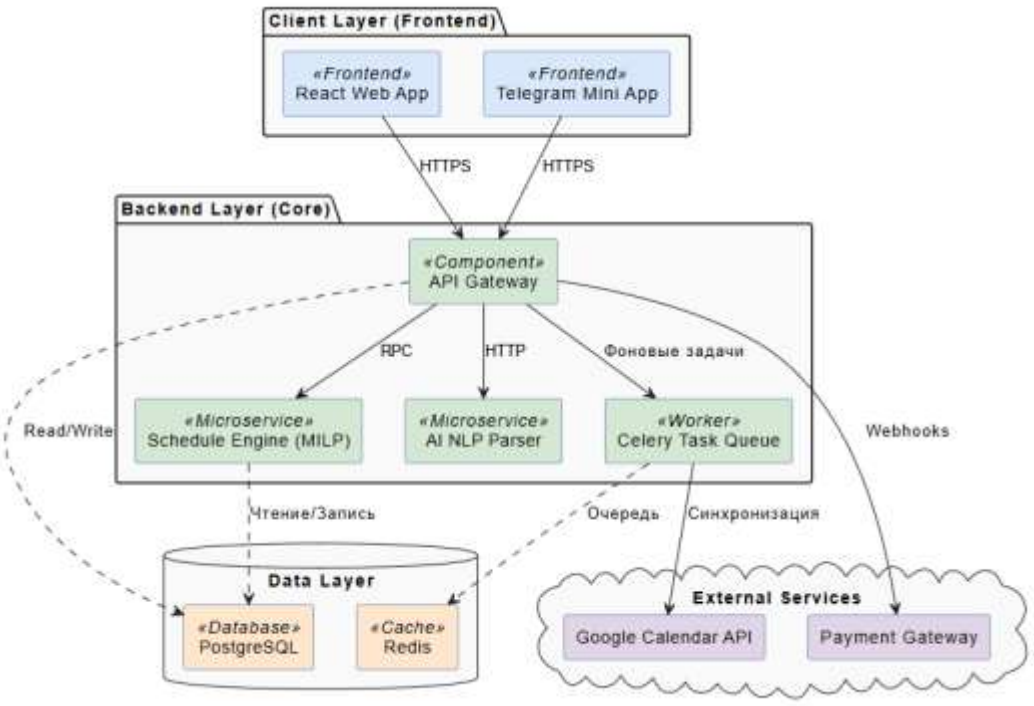


Рис. 6. Диаграмма компонентов (Component diagram) физической микросервисной архитектуры системы

Подробная расшифровка архитектуры (Слоев):

1. Client Layer (Слой того, что видит клиент - Фронтенд):

- **React Web App:** Версия планировщика для браузера на компьютере.
- **Telegram Mini App:** Мини-приложение, которое открывается прямо внутри мессенджера Telegram.

2. Backend Layer (Слой "Мозгов" на сервере):

- **API Gateway (API-шлюз):** Главный регулировщик (охранник) на сервере. Принимает абсолютно все запросы от пользователей, проверяет пароли и раздает команды нужным отделам.
- **Schedule Engine (Движок планирования):** Отдельная микропрограмма, которая занимается только математикой (тем самым MILP-алгоритмом).
- **AI NLP Parser (ИИ-парсер):** Отдельная микропрограмма с подключенной нейросетью для чтения текстов.
- **Celery Task Queue (Очередь фоновых задач):** Рабочий-невидимка. Если нужно сделать что-то долгое (например, отправить 1000 уведомлений в Telegram), чтобы приложение не "висело", эта работа отдается Селери, и он делает её тихо в фоне.

3. Data Layer (Слой хранения данных):

- **PostgreSQL:** Главная база данных. Строгая и надежная. Тут лежат пароли, списки задач и настройки пользователей.
- **Redis:** Очень быстрая, но временная база данных (кэш). Нужна для работы фонового рабочего (Celery), чтобы он помнил, какие задачи ему поручили выполнить.

4. External Services (Внешние чужие сервисы):

- **Google Calendar API:** Сервис Гугла. С ним приложение общается, чтобы вытянуть пары из универа.
- **Payment Gateway:** Платежная система (например, ЮKassa), чтобы брать деньги за Pro-версию.

Как они общаются (Расшифровка стрелочек):

- **HTTPS:** Стандартный интернет-протокол. По нему телефон общается с сервером.

- **RPC (Remote Procedure Call):** Очень быстрый внутренний язык. По нему "мозги" на сервере общаются между собой (API шлюз передает математику в Движок планирования).
- **Webhooks (Вебхуки):** СМС-оповещения от чужих серверов. Например, платежная система сама "стучится" в API шлюз по вебхуку и говорит: "Пользователь только что оплатил подписку, дайте ему доступ".

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Вопросы к Диаграмме Вариантов Использования (Лаб. №2):

1. **Из каких элементов состоит диаграмма ВИ?**
   Диаграмма состоит из акторов (пользователей или внешних систем), прецедентов/ВИ (овалов, описывающих функции), границ системы (рамки) и отношений/связей между ними (ассоциации, включения, расширения, обобщения).
2. **В чем отличие отношений ВИ «Включение» (<<include>>) и «Расширение» (<<extend>>)?**
   Отношение <<include>> означает *обязательный* вызов: базовый ВИ не может выполниться без включаемого.
   Отношение <<extend>> означает *опциональный* вызов: расширяющий ВИ срабатывает только при выполнении определенного условия в базовом ВИ.
3. **Почему для создания диаграмм ВИ желательно использовать специализированные редакторы?**
   Они обеспечивают стандартизацию элементов по нотации UML, автовыравнивание, простоту редактирования связей (без необходимости перерисовывать всё вручную) и позволяют интегрировать схемы в системы документирования.

Вопросы к Диаграммам Последовательности и Компонентов (Лаб. №4):

1. **Для чего создается диаграмма последовательности?**
   Для моделирования логики взаимодействия объектов Системы строго во времени (в хронологическом порядке) в рамках выполнения одного конкретного сценария (прецедента).
2. **Скольким вариантам использования соответствует диаграмма последовательности?**
   Обычно одна диаграмма последовательности соответствует одному (иногда одному альтернативному сценарию) конкретного варианта использования.

3. **Что может выступать в качестве объектов на диаграмме последовательности?**
   Экземпляры классов, компоненты программного обеспечения, внешние системы, акторы (пользователи), интерфейсы и базы данных.

4. **Для чего создается диаграмма компонентов?**
   Для визуализации физической архитектуры (развертывания) системы, разбиения её на модули/библиотеки/артефакты и показа интерфейсов/зависимостей между ними.

5. **С помощью каких элементов на диаграмме можно показать классы объектов?**
   В диаграмме последовательности объект показывается прямоугольником с подчеркнутым именем вида имя_объекта : ИмяКласса. В диаграмме компонентов — прямоугольником со значком компонента.

6. **Как на диаграмме можно показать экземпляры классов объектов?**
   Аналогично п.5, имя экземпляра пишется перед двоеточием с подчеркиванием. Например: myTask : Task.

7. **Что такое порт и для чего он нужен?**
   Это специфическое "окно" в инкапсулированном компоненте (квадратик на границе), через которое компонент принимает и отправляет сообщения, связывая свои внутренние части с внешней средой через интерфейсы.

8. **Что такое линия жизни объекта и что она обозначает?**
   Вертикальная пунктирная линия под объектом на диаграмме последовательности. Она обозначает период времени, в течение которого объект существует в системе.

9. **Как могут быть созданы объекты?**
   Объекты могут быть созданы в процессе взаимодействия. Это показывается стрелкой с сообщением <<create>>, направленной прямо в прямоугольник нового объекта, который рисуется не на самом верху, а в момент его создания на временной шкале.

10. **Каким образом располагаются объекты в схеме и почему?**
    Объекты располагаются горизонтально слева направо. Крайним слева обычно располагается инициатор взаимодействия (Актор), а далее объекты располагаются по мере их вовлечения в процесс передачи сообщений.

11. **Что такое коннектор и какими виды связей предполагаются к использованию в коннекторах?**
    Коннектор — это линия соединения между двумя портами или компонентами. Связи могут быть обычными зависимостями (линия) или связями через интерфейсы (нотация "шарик-гнездо" / требуемый-предоставляемый интерфейс).
12. **Что такое временная шкала, для чего она нужна и как она располагается на схеме?**
    Временная шкала на диаграмме последовательности подразумевается вертикально, направлена сверху вниз. Она нужна для отображения хронологического (последовательного) порядка отправки и получения сообщений между объектами.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены теоретические основы и получены уверенные практические навыки объектно-ориентированного проектирования с использованием языка UML.

На базе технического задания и пользовательских историй (из предыдущих лабораторных работ) была разработана комплексная архитектура интеллектуального планировщика задач Flowra.
Использование **поведенческих диаграмм** (Use Case, Activity, Sequence, State Machine) позволило детально алгоритмизировать логику автопланирования (MILP), определить жесткие и опциональные зависимости бизнес-процессов, а также проследить жизненный цикл задачи в системе.
Использование **структурных диаграмм** (Class, Component) обеспечило плавный переход от схемы базы данных к объектно-ориентированному коду: были спроектированы классы с соблюдением принципов инкапсуляции, определены точные типы связей (агрегация и композиция) и выстроена физическая микросервисная архитектура приложения.

Разработанный пакет UML-моделей полностью соответствует стандартам разработки ПО и является готовой основой для старта этапа программирования системы. Цели и задачи лабораторной работы достигнуты в полном объеме.